

Metodologie di Programmazione (A-L)

Il semestre - a.a. 2025 – 2026

Incapsulamento e UML

a cura di Massimo La Morgia



SAPIENZA
UNIVERSITÀ DI ROMA

*Tutti i diritti relativi al presente materiale didattico ed al suo contenuto sono riservati a Sapienza e ai suoi autori (o docenti che lo hanno prodotto). È consentito l'uso personale dello stesso da parte dello studente a fini di studio. Ne è vietata nel modo più assoluto la diffusione, duplicazione, cessione, trasmissione, distribuzione a terzi o al pubblico pena le sanzioni applicabili per legge.

**I crediti sulle slide di questo corso sono riportati nell'ultima slide

Incapsulamento

- Perché utilizziamo le parole chiave `public` e `private`?
- Per nascondere le informazioni ("information hiding") all'utente
- Il processo che nasconde i dettagli realizzativi, rendendo pubblica un'interfaccia, prende il nome di incapsulamento
 - Dettagli realizzativi: campi e implementazione
 - Interfaccia pubblica: metodi pubblici

Perché incapsulare?

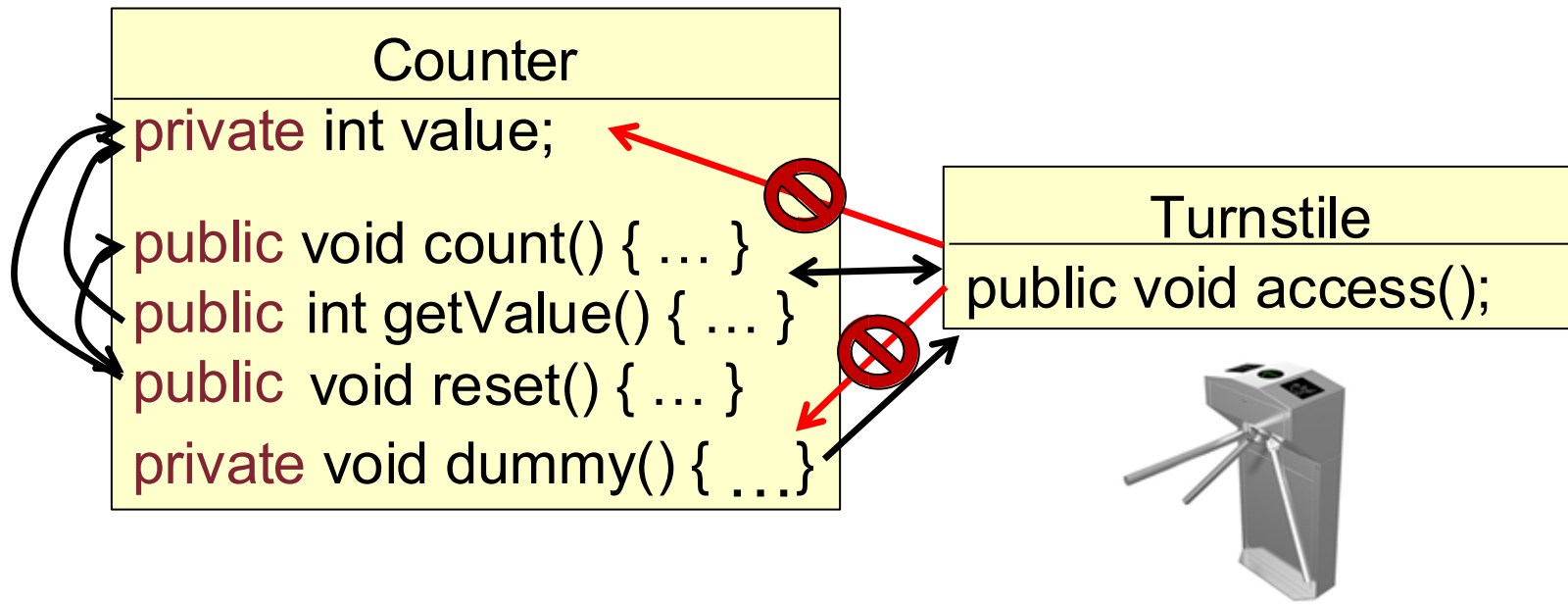
- Si semplifica e modularizza il lavoro di sviluppo assumendo un certo funzionamento a "scatola nera"
- Non è necessario sapere tutto, soprattutto molti inutili dettagli
- L'incapsulamento facilita il lavoro di gruppo e l'aggiornamento del codice (maintenance)
- Aiuta a rilevare errori: in presenza di moltissime classi, un certo errore si verifica solo in una determinata classe per cui ci si può concentrare su di essa

Come interagiscono le classi tra loro?

- Una classe interagisce con le altre **principalmente (=quasi solo)** attraverso i costruttori e i metodi pubblici
- Le altre classi non devono conoscere i dettagli implementativi di una classe per usarla in modo efficace

Accesso a campi e metodi (inclusi i costruttori)

- I campi e i metodi possono essere **pubblici**, **privati** (o **protetti**, come vedremo più in là)
- I metodi di una classe possono chiamare i **metodi pubblici e privati della stessa classe**
- I metodi di una classe possono chiamare i **metodi pubblici** (ma non quelli privati) di altre classi



UML parte 1: i diagrammi delle classi



WIKIPEDIA
L'enciclopedia libera

Accesso non effettuato [discussioni](#) [contributi](#) [registrati](#) [entra](#)

Voce [Discussione](#)

Leggi

[Modifica](#)

[Modifica wikitesto](#)

[Cronologia](#)

Cerca in Wikipedia



Class diagram

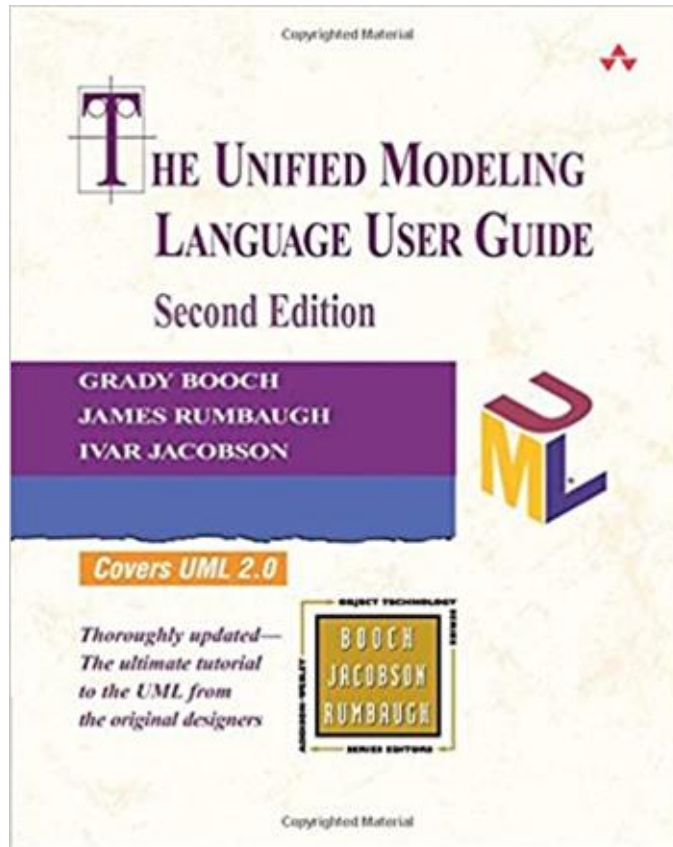
Da Wikipedia, l'enciclopedia libera.

I *diagrammi delle classi* (**class diagram**) sono uno dei tipi di [diagrammi](#) che possono comparire in un [modello UML](#). In termini generali, consentono di descrivere *tipi di entità*, con le loro caratteristiche e le eventuali relazioni fra questi tipi. Gli strumenti concettuali utilizzati sono il concetto di [classe](#) del [paradigma object-oriented](#) e altri correlati (per esempio la [generalizzazione](#), che è una relazione concettuale assimilabile al meccanismo *object-oriented* dell'[ereditarietà](#)).

In [ingegneria del software](#), **UML** (*Unified Modeling Language*, "linguaggio di modellizzazione unificato") è un [linguaggio di modellazione](#) e di [specificazione](#) basato sul [paradigma orientato agli oggetti](#). Il nucleo del linguaggio fu definito nel 1996 da [Grady Booch](#), [Jim Rumbaugh](#) e [Ivar Jacobson](#) (detti "i tre amigos") sotto l'egida dell'[Object Management Group \(OMG\)](#), consorzio che tuttora gestisce lo standard UML.



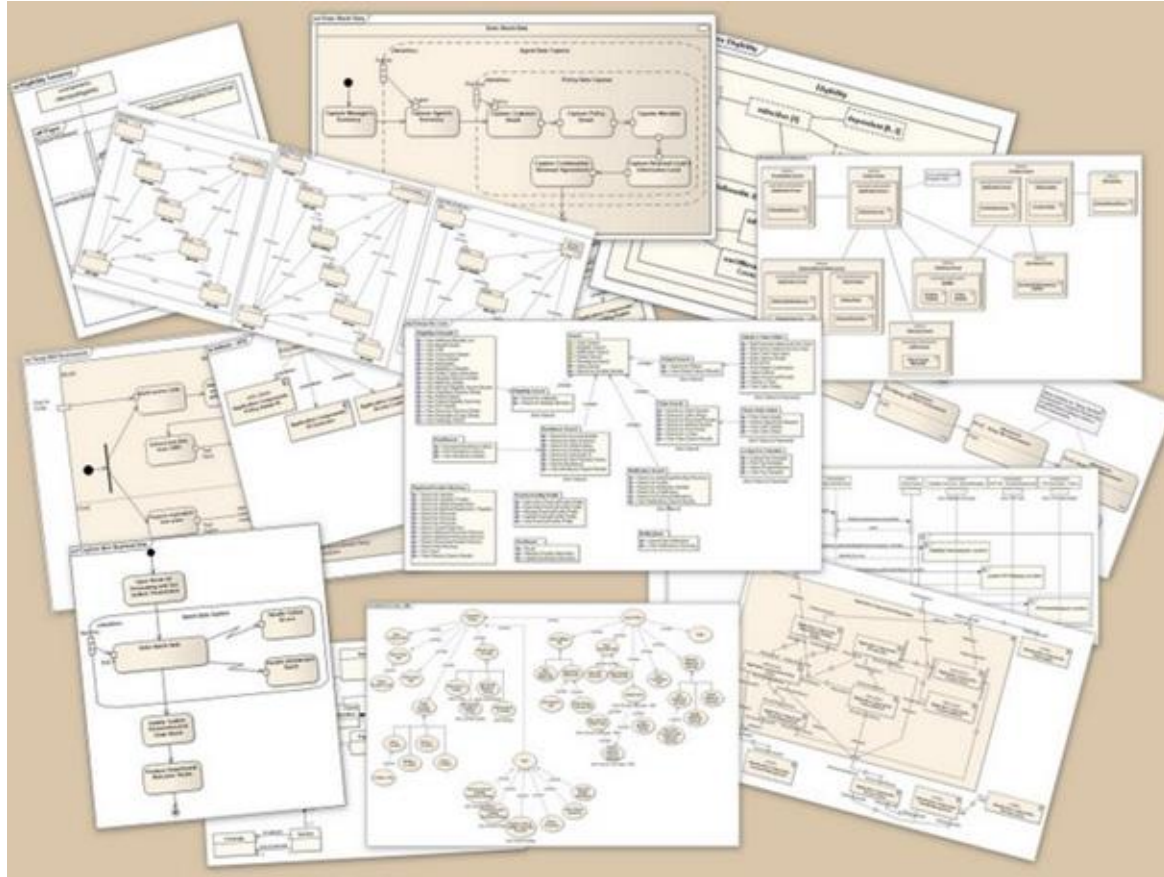
UML parte 1: i diagrammi delle classi



https://personal.utdallas.edu/~chung/Fujitsu/UML_2.0/Rumbaugh--UML_2.0_Reference_CD.pdf



UML parte 1: i diagrammi delle classi



UML parte 1: i diagrammi delle classi

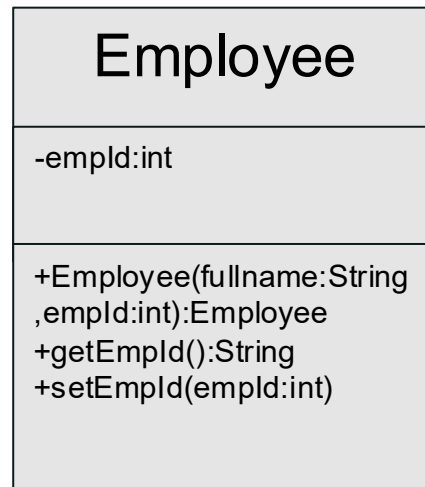
Object Oriented Analysis

Reverse Engineering

Entità
(Oggetti nel
mondo reale)



Classe in UML
modello



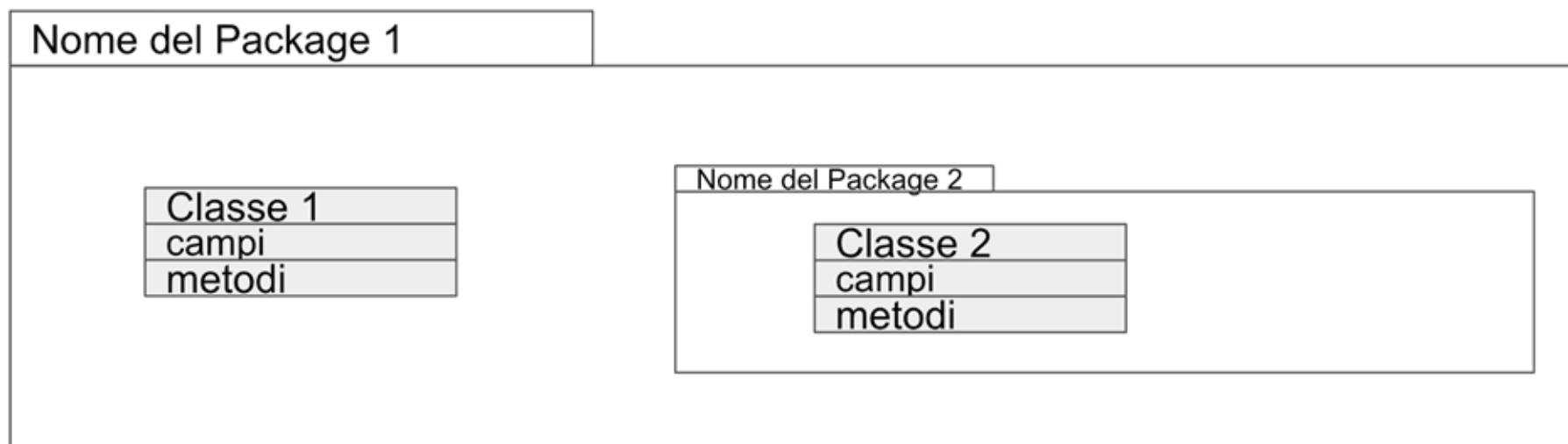
Classe in
Java

```
1 // Java bean for Employee
2 public class Employee extends Person {
3     // Private variable
4     private int empId;
5     // Constructor
6     public Employee(String fullName, int empId) {
7         super(fullName);
8         setEmpId(empId);
9     }
10    // Getter and setter for variable
11    public int getEmpId() {
12        return empId;
13    }
14    public void setEmpId (int empId) {
15        this.empId=empId;
16    }
17 }
```

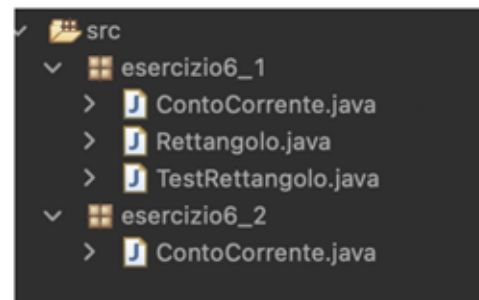
Oggetti in
Java

```
new Employee("John",...);
new Employee("Dessy",...);
```

UML parte 1: i diagrammi delle classi



In questo Diagramma abbiamo un package che include una classe e un sotto package



UML parte 1: i diagrammi delle classi

```
23 public class Rettangolo {
24     private double x, y, lunghezza;
25     private double altezza;
26
27     public Rettangolo(double x, double y, double lunghezza, double altezza)
28     {
29         this.x=x;
30         this.y=y;
31         this.lunghezza=lunghezza;
32         this.altezza=altezza;
33     }
34
35     public void trasla(double x, double y)
36     {
37         this.x+=x;
38         this.y+=y;
39     }
40
41     public String toString()
42     {
43         double x2=x+lunghezza;
44         double y2=y+altezza;
45         return "("+x+", "+y+")-> ("+x2+", "+y2+")" ;
46     }
47 }
```

Rettangolo

-x:double
-y:double
-lunghezza:double
-altezza:double

+ Rettangolo(x:double,...):Rettangolo
+ trasla(x:double, y:double)
+ toString():String

UML parte 1: i diagrammi delle classi

- + (visibilità pubblica): ogni elemento che può accedere alla classe può anche accedere a ogni suo membro con visibilità pubblica
- (visibilità privata): solo le operazioni della classe possono accedere ai membri con visibilità privata
- # (visibilità protetta): solo le operazioni appartenenti alla classe o ai suoi discendenti possono accedere ai membri con visibilità protetta
- ~ (visibilità package): ogni elemento nello stesso package della classe (o suo sottopackage annidato) può accedere ai membri della classe con visibilità package



Un esempio: realizzare un semplice menù

- Dovete realizzare la classe `Menu`, in grado di visualizzare un menù come questo:

- 1) Inizia il gioco
- 2) Carica gioco
- 3) Aiuto
- 4) Esci

Fase 1: identifica i metodi richiesti

- Aggiungere una nuova opzione
- Visualizzare il menù

Fase 2: specifica l'interfaccia pubblica

- Aggiungere una nuova opzione:

```
public void addOption(String option) { }
```

- Visualizzare il menù:

```
public void display() { }
```

- Costruire l'oggetto:

- Costruttore con una prima opzione in input?
- Costruttore vuoto?

- Meglio evitare casi speciali:

```
public Menu() { }
```


Fase 3: scrivere la documentazione per l'interfaccia pubblica

```
/**
 * Un menu' che viene visualizzato in una finestra di console
 * @author navigli
 */
public class Menu
{
    /**
     * Costruisce un menu' vuoto (senza opzioni)
     */
    public Menu()
    {
    }

    /**
     * Aggiunge un'opzione alla fine del menu'
     * @param option l'opzione da aggiungere
     */
    public void addOption(String option)
    {
    }

    /**
     * Visualizza il menu' su console
     */
    public void display()
    {
    }
}
```

Fase 4: Identifica i campi

- In ogni momento, qual è lo stato di un oggetto di tipo **Menu**?

```
public class Menu
{
    private String menuText;
    private int optionCount;

    ...
}
```

Fase 5: Implementa i metodi

```
public class Menu
{
    ...

    /**
     * Costruisce un menu' vuoto (senza opzioni)
     */
    public Menu()
    {
        menuText = "";
        optionCount = 0;
    }

    /**
     * Visualizza il menu' su console
     */
    public void display()
    {
        System.out.println(menuText);
    }

    /**
     * Aggiunge un'opzione alla fine del menu'
     * @param option l'opzione da aggiungere
     */
    public void addOption(String option)
    {
        optionCount++;
        menuText += optionCount + ") " + option + "\n";
    }
}
```

Fase 6: Collauda la classe

- Scriviamo un'altra classe per "testare" (collaudare) la nostra:

```
public class MenuTest
{
    static public void main(String[] args)
    {
        Menu menu = new Menu();

        menu.addOption("Open new account");
        menu.addOption("Log into existing account");
        menu.addOption("Help");
        menu.addOption("Quit");

        menu.display();
    }
}
```

Esercizio 1/2

- Progettare una classe **Programmatore** i cui oggetti rappresentano persone che svolgono il lavoro di sviluppo presso un'azienda
- La classe implementa i seguenti metodi:
 - un costruttore con il nome e cognome della persona
 - un metodo **setAzienda** che imposta il nome dell'azienda per cui la persona lavora
 - un metodo **addLinguaggio** che aggiunge un linguaggio di programmazione a quelli inizialmente usati dal programmatore
 - i metodi **getNome**, **getCognome** e **getAzienda** che restituiscono i valori corrispondenti
 - un metodo **getLinguaggi** che restituisce la stringa dei linguaggi (separati da spazio) noti al programmatore

Esercizio 2/2

- Implementare un main all'interno della stessa classe in modo che il seguente codice funzioni correttamente:

```
Programmatore p1 = new Programmatore("Bjarne", "Stroustrup");
Programmatore p2 = new Programmatore("Brian", "Kernighan");
Programmatore p3 = new Programmatore("James", "Gosling");

p1.addLinguaggio("C");
p1.addLinguaggio("C++");
p1.setAzienda("Morgan Stanley");

p2.addLinguaggio("C");
p2.addLinguaggio("AWK");

p3.addLinguaggio("Java");
p3.setAzienda("Oracle");

// stampa: Morgan Stanley
System.out.println(p1.getAzienda());
// stampa: C AWK
System.out.println(p2.getLinguaggi());
```

Credits

Le slide di questo corso sono il frutto di una personale rielaborazione del Prof Faralli, che sono a loro volta una rielaborazione delle slide del Prof. Navigli.

In aggiunta, le slide sono state revisionate dagli studenti borsisti della Facoltà di Ingegneria Informatica, Informatica e Statistica: Mario Marra e Paolo Straniero.

Font ad alta leggibilità Biancoenero® di biancoenero edizioni srl, disegnata da Umberto Mischi, con la consulenza di Alessandra Finzi, Daniele Zanoni e Luciano Perondi (Brevetto n. RM20110000128).

Disponibile gratuitamente per tutte le istituzioni e i privati che la utilizzino per scopi non commerciali.